SaltBench v1: A Referee-Gated, Pre-Registered Protocol for Measuring Method Effects in Machine-Checked Software Work, with Frontier Baselines on Lean and Verus

Jason Hickey

September 2026
version 1, the protocol and the baselines; version 2 will carry the custom populations

Abstract

SaltBench is a benchmark protocol for one question: does a change to how a coding agent is instructed change what it can get past a machine referee? The protocol fixes four things before any model call. A referee decides every outcome (a proof kernel, a program verifier, or a hidden test suite), and the agent's work is re-checked outside its own elaboration. A fence denies the agent the network, the ground truth, and the harness state, and the fence is measured by probes before the run rather than assumed. Every run is authorized by a dated freeze commit, with predictions registered and scored, adverse outcomes named, and later changes appended as amendments that never edit the frozen text. A budget stop is a halt, never a failure. We report the protocol, three task populations (CLEVER Task 1 in Lean 4, VeruSAGE-Bench proof targets in Verus, and five systems components authored for this benchmark in Rust with withheld suites), and frontier baselines. The plain arm saturates the hard band of the Verus population at Opus 5 (9 of 10, with the turn cap binding in the shoulder), and SWE-bench Verified at Sonnet 5 (13 of 15 on both control arms, the two failures at the call cap, with contamination present and unquantifiable). On the Lean population the pre-registered comparison of a plain arm, an equal-length placebo arm, and the treatment arm returned a null at Sonnet 5 (8 of 15 on every arm, and the treatment arm matched the control problem for problem), and a blind triage of the failures attributes most of them to the reference oracle rather than to the agent, including one problem whose reference specification is machine-checked unsatisfiable. On the authored population, measured for cost rather than for capability at Opus 5, a cross-problem sign test registered before the first cell returned a cost premium for the treatment arm on all five problems, one-sided $p = 0.0312$. Four qualifiers were registered with that test and travel with it wherever the p-value goes: the test is one-sided and cannot significantly refute the hypothesis it tests; three of the five per-problem magnitudes fall below the design's own resolvable floor of $2.0072\times$ at $n = 3$, and two resolvable magnitudes out of five are not a magnitude for the population, so the finding is a sign and no ratio may be reported as the result; the paired arm the campaign most wanted is buildable on one problem only, which is $k = 1$ and reaches no verdict at any outcome; and the within-condition spread on one arm of one problem is $1.80\times$, larger than the smallest premium. No cell of that run carries a referee verdict on a withheld suite, so it is a cost result with correctness unmeasured, and no sentence in this paper pairs its premium with working code. We report the instrument findings as results in their own right: a cap measured from data the cap produced returns the cap; a gate can penalize the treatment for applying the treatment; a screen's false positives are invisible until someone goes back for the refused cell; a fence probed only in its sandbox's language cannot see the layer above it, and no scored episode of this campaign had the agent's own file tool fenced by path, though across

278 landed Lean episodes no agent ever directed a file tool at a fenced path. No effect of the treatment on what a referee accepts is claimed anywhere in this paper. The tests that could show one are stated and remain open.

1 Introduction

A benchmark for agent methods needs two properties that usually pull against each other. The outcome has to be decided by something the agent cannot argue with, and the comparison has to be planned before the agent runs, so that the result cannot be narrated afterwards. SaltBench is a protocol that insists on both. The referee is a proof kernel, a program verifier, or a hidden test suite, run outside the agent's own toolchain invocation. The comparison is pre-registered [10]: the population is drawn by a committed seed, the arms are byte-pinned, the predictions are written down, the adverse outcomes are named, and the freeze commit is the authorization. Everything that happens afterwards is a dated amendment appended to the record.

This paper is version 1. It claims the protocol, three populations with their provenance, frontier baselines on those populations, a pre-registered null on one of them with a triage that explains most of it, a pre-registered cost reading on the third, and a set of instrument findings that we think transfer to any benchmark of this kind. It does not claim an effect of the method under test on what a referee accepts. Section 6 states the tests that could produce one; they are the content of version 2.

The method under test is a set of working practices for machine-checked development that the author uses in a formal mathematics project. Its rendering as an arm is described in Section 2.6; only the part of it that a single sealed agent can carry is rendered, and that limit was registered before the first call.

2 The referee-gated protocol

2.1 The referee decides, outside the agent's elaboration

An episode ends when the agent stops or hits a cap. The file it leaves is then scored by a checker the agent never sees, on a machine the agent never reaches. What "solved" means is fixed per substrate.

Lean (CLEVER [14]). The checker extracts only the agent-writable bodies by section markers, screens them for command-introducing and meta keywords with comments and string literals stripped, assembles a canonical file from the frozen statements and the bodies, and compiles it under an operating-system sandbox. It then replays every declaration of the compiled module through the Lean kernel [5] (`Lean.Environment.replay`), so a declaration the elaborator admitted without the kernel is rejected. It compares, as kernel expressions, the type of every frozen declaration and the value of the human specification between the canonical file and a pristine file with `sorry` bodies, so a notation, macro, instance or `open` that changes what the frozen text means is caught by construction. It collects the axioms of the audited declarations and requires every set to be inside {`propext`, `Classical.choice`, `Quot.sound`}; `sorryAx`, user axioms, and `native_decide`'s `Lean.ofReduceBool` all fail here. The class names the first failing gate: `SCREEN`, `COMPILE`, `KERNEL_REJECTED`, `STATEMENT_ALTERED`, `AXIOMS_FAIL`, `PASS`.

Verus [8] (VeruSAGE-Bench [15]). Verus emits no proof object and there is no independent re-check, so the protocol says "the referee accepted" where the Lean protocol says "kernel". The

integrity work that the kernel replay and axiom audit do in Lean is done here by three text and AST layers before the referee runs: a screen over the agent's regions; a count guard over `admit()`, `assume(`, `external_body` in both spellings, `assume_specification` and `verifier::admit`, canonical against original; and the benchmark's own `lynette additions` AST comparison. A screened or cheating body is never handed to the referee. `PASS` requires exit 0 and a results line that parses as `N verified, 0 errors` with $N \geq 1$ and no partial-verification suffix. Resource-limit exhaustion and compile errors are their own classes and are never charged to the arm.

Rust (S3-Systems). The referee for the authored population is the withheld test suite of the card, run by the harness's own runner outside the agent's workspace, with the suite and its mutant set never shipped to the agent. That is the design, and this paper reports one run on this population (Section 4.5) in which the referee was never invoked: the cells were scored for cost and not for outcome, and no cell carries a verdict. The paragraph is here rather than omitted because a reader who meets three referees in this section and four populations in the next will otherwise supply the missing one from the pattern, which is exactly the inference the run does not support.

SWE-bench [7] Verified [11]. The scorer is the unmodified `swebench` harness in the pinned evaluation images, with the fail-to-pass and pass-to-pass suites hidden from the agent. This substrate was abandoned after stage 0 (Section 4.1); its protocol is kept in the record.

2.2 The fence is measured, not assumed

The agent's subprocesses run under the operating system's sandbox (macOS Seatbelt through the agent harness's own sandbox settings): every host denied, writes only under the working copy and the session temp directory, reads denied on the harness state, the credential trees, and the run root. The checker runs under its own `sandbox-exec` profile. A command-text hook on the shell tool is kept as an attempt audit, not as the fence, and an audit layer records every tool read by path. Five smoke probes run before the driver starts: a canary, network escapes (including an in-language process spawn through the toolchain wrapper), reads of the arm files and the harness config, writes through the symlinked build directory, and the compile path. The driver refuses to start unless the last verdict of every probe is a pass carrying the freeze's episode-script hash.

Ground truth never reaches the host while an arm writes a specification blind. The stage-A views carry no human specification; the stage-B and stage-C views are shipped only after every stage-A episode has landed, and the ship is verified by a content set-hash rather than by the exit code of the tool that moved it.

Two findings in the record show why the fence is measured. The Verus referee's docstring claimed a sandbox, a process group and a belt; a grep of the file found only the comment, and the referee had been running unfenced. The exec set turned out to be four binaries, established by a ladder in which each arm's own error names the next binary, and a fifth exec point was revealed only by the red arm's panic text. A proof-only fixture had passed the four-binary fence, which would have been the receipt.

The third finding arrived after every read reported in this paper and applies to all of them. The sandbox's read denials bind sandboxed subprocesses: the shell tool and whatever it executes. The agent harness's own file-reading tool is executed by the harness process and never enters that sandbox. Neither substrate's settings carried a tool-permission rule, so no scored episode of this campaign had the agent's tools fenced by path. The smoke probe that had certified credential reads denied made its reads through the shell and through an in-language process spawn, both sandboxed; a probe written in the sandbox's language cannot see a hole in the layer above it, and

it reads afterwards as though the agent itself were fenced. What was in force, each with its own evidence, was the subprocess sandbox, the empty network allowlist, the shell-tool hook, the ground-truth leak check, and the audit layer. That audit is what establishes the S2-Rust read: across its 15 Opus episodes no episode was voided and no unblocked read of a fenced path was recorded (one attempt through the shell was blocked). One episode of the lower-tier probe reported in Section 6 directed its file tool at a path under its own configuration directory, was not refused, and was voided as unscorable; read at the transcript the call returned that the file did not exist, so no bytes came back, and the audit field had recorded that the call was not blocked rather than that it was served. Those are different facts, and only one of them is a fence event. Over every landed S2-Lean episode in the campaign's four state roots, 278 with a transcript and 278 parsed, the count of file-tool calls at a fenced path is zero served and zero not served: no agent ever directed a file tool at a path under a deny root. The detector was driven on all three branches (a served read of a fenced path counted, a refused read classified and not counted, a read of an unfenced path ignored), because a quiet failure reads as good news. What this measures is what the agents did, not what they could have done: the canary shows the gap was open, and an absence of exploitation is not a presence of protection. The repaired fence derives a tool-permission deny list from the same path list as the sandbox denials, 132 rules beside 11 paths, and was driven with a canary in both arms: with the tool layer absent the canary reached the agent's final message, and with it present the agent reported the file unreadable and the canary appears in neither the result nor the transcript.

2.3 Pre-registration, and the amendment discipline

The dated freeze commit is the authorization. Before it: the population rule and seed, the arms as byte-pinned files, the checker, the constants, the reading rule, the predictions, the adverse outcomes, the stop rules with their enforcer. After it: nothing above the line is edited. Every later change, including a repair to an instrument, is a dated amendment appended before its own first model call, with its own predictions. Corrections to a registered document are appended as corrections, never edited away; the record carries several such corrections by its own authors.

The reading rule on the Lean and Verus populations is a count, not a p-value: for two arms on the same drawn problems, b is the number the first arm proves and the second does not, c the reverse, and $|b - c| < 5$ is read as INDISTINGUISHABLE. The threshold is registered before the first call and the outcome table is enumerated, with the adverse cell named. For the Verus wave the fixed separation is replaced by one derived from a measured null (a same-arm rerun), because the substrate's own paper [15] reports that 11.8 % of failures flip on rerun.

Predictions are scored as they stand. Of the registered predictions in the record, several failed, and the failures are recorded as failures with their direction: the estimator over-priced five consecutive stages, then under-priced the sixth because one episode carried 64 % of a stage's spend.

2.4 A budget stop is a halt, never a failure

A per-episode token stop at four times the governing regime's p90 is registered as a halt: a halted episode is recorded as halted and is unresolved for the rate, because scoring an episode the budget stopped as a failure would let the budget instrument move the result. The multiple was chosen from the record: no passing episode among 177 had exceeded 2.40× its regime's p90, and the one runaway was at 7.79× and failed.

The rule had to be enforced where the verdict is written, not only where it is registered. The first halted Verus episode was scored `SCREEN` on a snapshot of an interrupted edit (two `assume` calls the agent had not yet discharged). The checker now takes the episode's termination and writes

`class = HALT` for a token or wall stop, preserving the class it would otherwise have written as a diagnostic.

2.5 Ask of every gate which arm is more likely to trip it

The treatment arm encourages helper lemmas. Three gates in the Verus grader were found, in sequence, to refuse exactly that. The AST comparison refused a helper placed before the enclosing `impl`, classifying the episode as `STATEMENT_ALTERED`, a cheating class, for doing what the prompt invites; 47 of 207 views are impl-enclosed. The screen's rule set enforced 29 refusals of which the prompts named 12, and the unstated `use` rule bites in the helpers region; the repair makes the prompt a consumer of the rule set, with a gate that refuses a rule with no prompt entry. The helpers whitelist then split items by brace depth and refused a legitimate lemma whose `ensures` clause carried struct literals; it was the third episode ever run that found it, after 29 screen arms, 18 fence arms and an 11-arm fixture kit had passed. An instrument that penalizes the treatment for applying the treatment does not measure a small effect badly; it manufactures the opposite one. The discipline is now a standing question at every gate.

2.6 The arms

Every arm receives the same task file and the same stage prompt; only the agent's instruction file differs, and it is byte-pinned. `a0` is the plain arm: the base block alone. `a1` is the placebo: the base block plus an equal-length process prompt with no method content (1,746 bytes against the treatment's 1,913; the base is 627). `a2` is the treatment: the base block plus the method's solo-renderable core. The rendering, article by article against its source, and the articles that are not rendered because they presuppose an organisation with a human in it, were registered before the first call; no result from `a2` may be read as a test of the method's multi-agent tier. The treatment text names `sorry`, which is the control's dominant failure; the objection that the arm coaches to the metric was registered in advance and is answered by the data in Section 4.2.

3 Populations and provenance

S2-Lean: CLEVER Task 1. `trishullab/clever` [14] at commit `8348039`, MIT, Lean `v4.27.0` [5], mathlib `a3a10db0` [9]; 161 problems. A raw problem file is re-cut by the harness into three views: stage A (write `generated_spec` from the docstring, ground truth absent), stage B (prove `spec_isomorphism` between the agent's frozen specification and the revealed human one), stage C (implement and prove `correctness` against the human specification, tests included). The draw sorts the real ids by a committed seed after removing the four structural exclusions of the benchmark's agentic-proving paper [12]. Stage 0 ran $k = 30$. The registered comparison population is U15, the first fifteen unflagged ids of that order (the same paper flags 80 of 161 specifications), reached at depth 27. Stage C's population removes the two U15 ids whose stage-C view fails to elaborate before any agent text (problems 54 and 112) and one further id, problem 18, whose task is machine-checked unsatisfiable (Section 4.3), leaving 12.

S2-Rust: VeruSAGE-Bench proof targets. `microsoft/verus-proof-synthesis` [15] at commit `cbf9c0c6`, MIT; the Anvil-Advanced [13] and NRKernel [8] projects, 267 records. The wave takes the unique `proof fn` target whose body is empty modulo whitespace: 207 records; the five refuse classes are counted and published with the draw. Stage 0 then ran every reference proof through the pinned referee inside the harness's own scaffold: 180 live, 27 with a task text that

omits definitions its own ground truth supplies, zero dead tasks, zero pin defects, zero scaffold defects. The pin itself was measured: the current Verus release failed 5 of 15 reference proofs at the Rust front end, and the benchmark's own release passed 15 of 15; a front-end error on a reference file indicts the toolchain, never the task. The resource-limit curve is flat into the registered limit (179 of 180 at 10, 180 of 180 at 50 and at 250) and three repeats at the pinned seed produced zero flips.

S1: SWE-bench Verified. `princeton-nlp/SWE-bench_Verified` [11] at revision `c104f840`, 500 rows, content-pinned by a hash over the canonicalised rows. The selection script is the normative form of the criteria; the draw is measured-50 and pilot-30 with a per-repository cap of 9, chosen from arithmetic over the eligible counts after the first draft's cap starved the draw in silence.

No task text from SWE-bench Verified is redistributed. Its dataset card carries no licence field, so rather than rely on a reading of what the issue text permits, the repository ships the 30 identifiers, the pinned revision, a hash over the 500 canonicalised rows and a hash over the 30-row projection the episodes read; `harness/fetch_problem_statements.py` rebuilds that projection from the pinned revision and verifies it against both. The rebuild was compared byte for byte against the file the episodes read before that file was removed from the tree.

S3-Systems: five components authored for this benchmark. The third population is not drawn from a public set. Five classic systems components are authored for this benchmark in Rust and frozen in a dated export before any model call: `Crc32`, `FreeList`, `LRU`, `LZW` and `Paxos`. Each is frozen as four files of the same shape across all five: a task card, a greenfield reference (`G`), a base plus a change request for a maintenance card (`B`), and the referee's name list. The five problems' frozen material totals 9,141, 12,917, 14,643, 20,861 and 16,037 bytes across those files, and none of the five is a stub. Each of the ten cards carries a withheld test suite, and a withheld mutant set was authored against it. Because the population is original, there is no third-party licence to reconcile and no contamination proxy to compute against a public gold patch; the offsetting cost is that the population has no independent authorship, which Section 4.5 states as a limitation rather than leaves implicit.

The withheld suites were measured against the withheld mutants before any cell ran, by driving the same runner the referee drives rather than a second invocation of the test tool: over the ten cards, 44 of 44 mutants killed, none surviving and none unmeasured, and every reference passing its own suite. That number is a ceiling and not a strength, because the mutants were authored beside the tests, and it is a property of the suite rather than a verdict on any cell. Only `LZW`'s card carries a `## Statement` section; the other four were authored without one, so 24 of 24 statement-arm cell builds refused at the harness, which is the harness working and a content gap in the population.

Licences and attributions for every source, and the exact list of what this repository redistributes from each, are in `PROVENANCE.md`.

4 Results

4.1 SWE-bench Verified saturates at Sonnet 5 and is abandoned as a substrate

Stage 0 ran the two control arms on the first 15 tasks of the pilot draw at `claude-sonnet-5`, effort high, 40-call cap, in the pinned images. Pre-flight and gold controls passed 15 of 15 each; nothing was excluded.

arm	solved	metered p50	metered p90
`a0` plain	13/15	566,108	1,602,434
`a1` placebo	13/15	531,692	1,604,555

Both arms resolved the same 13 and failed the same 2; $b = c = 0$. The two failures are the two tasks on which both arms hit the 40-call cap (cap-bound episodes: plain 3, placebo 2), so 13 of 15 is a lower bound at this cap, as the Verus band's 9 of 10 is. Five of fifteen pairs shipped byte-identical patches across arms. The pre-registered contamination proxy (edit similarity of the submitted patch to the gold patch, cut at 0.80, stated before computing) read 6 of 13 resolved tasks as high-similarity, which the rule classifies as INDETERMINATE. An unregistered blind panel found, and the record verifies at the transcript, that on two unresolved tasks the agent wrote lines of the upstream fix as its own edit input before any read could have shown them; recall did not deliver either solve. The substrate is saturated at this tier for this scaffold, contamination is present and unquantifiable, and the treatment arms were never run on it. The datum stays in the record as the reason the campaign moved to referees that decide.

4.2 S2-Lean: a pre-registered null, a tier that moves both arms, and a placebo that moves with them

The control read. The plain arm at `claude-sonnet-5` on the first 15 of the draw at a 40-call cap proved the isomorphism on 2 of 15. Eleven of the thirteen failures compiled, replayed, kept their statements, and closed the proof with `sorry`. The benchmark's own reference checker, run over the same fifteen artifacts, agrees with ours on every problem (2 of 15). Raising the cap to 100 calls on the three round-capped episodes flipped two to proofs, one at 93 calls. On the registered U15 at a uniform 100-call cap the plain arm proved 8 of 15; the nine flagged problems in the first 15 of the draw were 0 of 9 at the control read's 40-call cap, and every one of them terminated voluntarily with turns in hand, so budget does not explain the zero. The flagged subset was not rerun at 100 calls, so the flagged against unflagged contrast is 0 of 9 at 40 against 8 of 15 at 100, with the better n on one side only.

The comparison. All three arms on U15 at `claude-sonnet-5`, 100 calls, stage A then stage B, arms alternating per problem:

contrast	b	c	reading
`a2` vs `a0`	0	0	INDISTINGUISHABLE; the treatment matched the control's set on all 15
`a1` vs `a0`	1	1	INDISTINGUISHABLE

Every arm proved 8 of 15. Seven of the treatment arm's fifteen episodes ended with `sorryAx` in `spec_isomorphism`, the identical failure on the identical set as the control; being told in plain terms that a placeholder is not a proof changed neither which problems were solved nor how they failed. Totals were flat (50.7M, 51.7M, 51.4M metered tokens for `a0`, `a1`, `a2`). The paired split was not: on the eight problems both arms proved, `a2` used 15.79M tokens against `a0`'s 20.76M (−24 %); on the seven both failed, 35.62M against 29.92M (+19 %). This is $n = 8$ and $n = 7$ and is reported as a paired observation, not an effect.

The tier. The same U15, the same checker, `claude-opus-5`, both content arms: `a0` 10 of 15 and `a2` 10 of 15, `a0`-only {112}, `a2`-only {4}, $|b - c| = 0$. The whole run cost 26.1M tokens against the Sonnet run's 108.9M; the three 100-call cap-runners that failed at Sonnet were proved at Opus in a fraction of the tokens (problem 141: 11,757,659 tokens failing, 180,396 passing). The tier raise is not monotone per problem: `a0` lost problem 4. The cost split's sign did not reproduce at Opus. One treatment episode at Opus had been refused by the pragma screen for a `set_option` line; a registered differential re-check over the frozen artifact found a complete kernel-checked proof with and without the line, so `a2` reads 11 of 15 as a diagnostic and the contrast becomes $|b - c| = 1$, still INDISTINGUISHABLE.

The placebo at Opus proved 9 of 15, a strict subset of the plain arm's ten, missing only 112. Its registered band for arm-independence was $[9, 11]$; it landed on the boundary, by one problem's margin, and the reading is that the tier's gain is a property of the tier and not of prompt content.

Stage C at the ceiling. On the registered twelve stage-C problems at `claude-opus-5`, the plain arm certified implementation and correctness proof on 12 of 12, at a median of 284,716 tokens and 12 calls per episode, in 57 minutes. The registered gate reads a count of 8 to 12 as a ceiling hold; at 12 the reachability bound $c \leq n - P_0$ is $c \leq 0$, so there is no problem left for a treatment arm to win and the treatment did not run. A registered probe of the four problems whose reference oracle is provably permissive found that not one of the twelve passes exploited the vacuous region.

4.3 The triage: most of the null belongs to the oracle

A blind triage of every failed stage-B cell in the record (31 cells: 21 at Sonnet across three arms, 10 at Opus across two) labelled each from the two specification texts and the proof alone, with tier, arm, class and termination withheld. Thirty of the 31 cells are triageable: 17 are HUMAN-LOOSE (the agent's specification is strictly stronger than the human one, with a kernel-checked witness of non-isomorphism recorded where one could be written), 5 are AGENT-WRONG, 8 are PROOF-HARD. The thirty-first was a screen refusal that never failed to prove anything (the cell of amendment 9) and sits outside the taxonomy. The fourth registered label, HUMAN-BUGGY (the agent's specification contradicts a flagged clause), has count 0 and is unreachable on U15 by construction, because U15 excludes the flagged ids. The label is a property of the problem: eight failing problems, eight single labels, no exceptions, so the arms fail for the same reason on the same problems.

The pattern under four of the five false obligations is one defect: the human specification guards its conclusion with a well-formedness hypothesis and says nothing outside it, while the agent's specification, asked for as a `Prop` over the same signature, is total. A guarded specification and a total one are never isomorphic, and stage B asks for an isomorphism.

Problem 18 is worse than loose. In Lean `v4.27.0`, `String.drop` and `String.take` return a `String.Slice`, and slice equality is structural, so the reference specification's occurrence test is false at a genuine occurrence and the specification forces the answer 0 where the benchmark's own test demands 3. The statement that no implementation satisfies the specification and passes the test is machine-checked with axioms exactly the allowlist and no `sorry`. The elaboration check that guards the population had marked the problem fine, because it measures whether the task can be stated, not whether it can be done. Within the registered population, problem 18 is the only carrier of this defect.

The audit of the benchmark's reference checker adds two findings. On non-adversarial artifacts it agrees with ours exactly. On adversarial ones it does not: its submission path rebuilds the problem and then compiles the solver's own view, so replacing the theorem to be proved with `True` certifies 13 of 15 with the solver's helper lemmas and 15 of 15 without them; and a helper lemma

`axiom cheat (P : Prop) : P` discharging the real goal certifies 15 of 15, because an axiom is not a `sorry` and the checker greps for the `sorry` warning. The axiom audit and the statement comparison in our checker exist for exactly these two routes.

4.4 S2-Rust: the plain arm saturates the hard band at Opus 5

Pilot. Three tasks drawn from the 180 live at seed 20260902, plain arm, `claude-opus-5`: 3 of 3 passed, metered 76,003 / 466,390 / 612,390, wall 1.1 to 4.8 minutes. The third pass is a re-score: the grader's helper whitelist refused the episode's legitimate helper lemma (the third arm-correlated instance, Section 2.5), the landing file records that refusal, and the verdict was re-scored from the archived agent output. Three of four registered predictions failed, all in the direction of over-pricing; a large task file is not a large task.

The hard band. Because the pilot's three tasks straddled the middle tercile and all passed, the upper tercile of the 180 live tasks by reference-proof wall (rank ≥ 120; the cutpoints are ranks, because absolute walls re-time 2.2 to 2.8 times faster on an idle machine while the order is stable) was registered as the band, with its 13 drawn ids, before any of it was spent on. In this benchmark difficulty and size are nearly the same axis: Spearman $\rho = 0.892$ between reference-proof wall and task bytes over the 180 live, and 42 of the 60 upper-tercile tasks are Anvil-Advanced.

With a registered sequential early stop, 10 of the 13 were run: 10 scorable, zero void, and **9 of 10 passed** at a 40-call cap; the stop fired at 10 because the ceiling threshold of 9 was already reached, saving 10,459,778 tokens at no loss of information. Spend was 22,424,889 against a 32,884,667 quote. The registered prediction was 5 to 9 with a point estimate of 7; the band held at its top edge and the point estimate was low by 2, the fifth consecutive under-estimate of the plain arm.

The cap sits in the shoulder. Uncensored passing call counts were 23, 26, 27, 27, 32, 35, 35, 39 against a cap of 40; two of ten episodes were at the cap, one of them a pass at exactly 40, and the one failure died at 40 calls reading eight verified and one error. A p90 computed from data the cap produced returns the cap. The measured 9 is therefore a lower bound on the ceiling, and the room for a treatment arm is smaller than the count shows. That a cap of this size can cut an episode which would otherwise have passed is now evidenced directly rather than inferred from the shoulder: at the lower tier on this same band, an episode that reported no discharged obligations at the 40-call cap carried a complete verified proof at call 79 when the cap was raised (Section 6).

The median episode spends 81 % of its tokens before its first clean referee run (range 64 to 91). Cost is dominated by search, not verification; a cap cuts search, not polish.

4.5 S3-Systems: a registered cost premium on all five problems, with correctness unmeasured

This is the only reading in this paper that returns a difference between the arms, and it is a difference in price and not in what a referee accepted. Everything in this section is a cost result, and the paragraph on correctness below states, and measures, that no cell of this run carries a correctness verdict at all.

The design, registered before the first cell. Five problems, four arms, $n = 3$ per condition, at `claude-opus-5` on the greenfield card. The arms are `plain-bare`, `diet-bare`, `plain-statement` and `diet-statement`. The treatment arm is a *dieted* rendering of the method: prescribed acts were removed after the full rendering hit the per-cell cost cap on a single earlier cell at $42.24,

and the registration requires the word *diet* on every headline, because this matrix does not measure the un-dieted method. The primary reading is a cross-problem sign test on premium(P) = median(diet-bare, P)/median(plain-bare, P), one premium per problem, against the null that each premium is equally likely either side of 1.0. The outcome table was enumerated before the first cell: 5 of 5 gives $p = 0.0312$, 4 of 5 gives $p = 0.1875$ and is registered in advance as *not* a positive result, 3 of 5 gives $p = 0.5000$. None of those numbers may be quoted on its own: the four qualifiers stated with the reading below were registered with the test and are part of it. Five problems is the smallest count at which a clean sweep can reach .05 at all, which is why the earlier two-problem stage could not have produced a verdict whatever its cells had said.

The client was pinned by absolute versioned path to the build the earlier stage had used, rather than resolved from the shell path, and the registration names the cost of that choice: the matrix measures a client two releases behind the one a path lookup would have fired. A symbolic link follows the newest install, so resolving a client from the path silently re-pins a scored run every time the vendor ships.

What was built, against what was registered. The registration and the campaign price cover 60 cells; 36 of them were buildable. (*Priced* is used in its own sense in the census below, where it means a cell that produced a cost, and the two senses are kept apart deliberately.) The 24 statement-arm cells on the four problems whose cards carry no `## Statement` section refused at the harness (Section 3), which leaves the bare pair complete on all five problems and the statement pair complete on `LZW` alone. The condition key is `(task, arm, card_extras)` read from each cell's own control directory, and a scorer keying on the arm alone would have pooled the bare and statement arms of the same treatment, doubled each apparent n, and mixed the very pair the design calls its gold, with no error raised and medians that look reasonable. The scorer that had scored every earlier stage correctly keys on two fields, because until this matrix a third was never needed.

The scoreboard.

problem	plain median	diet median	premium
Crc32	\$6.21	\$7.21	1.1610×
FreeList	\$13.77	\$37.60	2.7306×
LZW	\$13.95	\$19.18	1.3749×
Paxos	\$14.20	\$37.65	2.6514×
LRU	\$7.75	\$11.21	1.4465×

Cost is US dollars of metered subscription spend per cell, read from each cell's harvested `METER.txt`. Every premium in the third column is a ratio of medians at $n = 3$, and three of the five sit below the design's resolvable floor of 2.0072×. The column is printed so that the reader can reconstruct the sign, and by the registration no entry in it may be read as the finding.

The reading, and the four qualifiers that are part of it. Five of five premiums exceed 1. On the registered one-sided sign test that is $p = 0.0312$. The four qualifiers below were registered before the first cell fired, and the campaign's own rule is that they travel with the p-value wherever it is quoted; a reader who has the number without them has a different claim from the one this design supports.

1. **The test is one-sided.** It can confirm the hypothesis it tests and cannot significantly refute it: five premiums *below* 1 would have returned $p = 1.0000$. A one-sided test is a statement about

which surprise the design was willing to be surprised by, and this one measured the expected surprise.

2. **Every magnitude is unresolved.** At $n = 3$ with the registered pooled sd(ln cost) of 0.30458 the smallest resolvable premium is 2.0072×. Three of the five fall below it: Crc32 at 1.1610, LZW at 1.3749 and LRU at 1.4465. Only FreeList and Paxos clear it, and two resolvable magnitudes out of five are not a magnitude for the population. No ratio may be reported as the finding: not 2.7×, not a range, not a percentage. The finding is a sign across five problems.
3. **The paired arm the design calls its gold reaches no verdict.** It exists on `LZW` only, which is $k = 1$ and $p = 0.5000$ at any outcome. Its cells are plain-with-statement at \$10.24, \$11.39 and \$9.82 against diet-with-statement at \$22.53, \$15.34 and \$18.54, a ratio of medians of 1.8105×, and it carries no verdict.
4. **The within-condition noise is larger than the smallest premium.** `FreeList` on the plain arm spans \$13.02 to \$23.38 inside one condition on one problem, a spread of 1.80×, against a headline premium of 1.16× on Crc32. The registered spread clause says why the count of wide conditions is not itself a reading: at $n = 3$ with this dispersion a perfectly homogeneous condition crosses a 1.5× spread with probability 0.614, so about twelve of the matrix's twenty conditions are expected to look wide by sampling alone.

The declared set, and a dependency the scoreboard does not show. The scored set is declared by identifier rather than by a glob over the archive, because cells from different runs share the condition key and a glob pools them: an early draft of the scorer, driven on partial data, silently pooled this matrix with the earlier stage, with a pricing set, with two dropped arms and with a void cell. The declaration is the cells of the matrix root plus three named smoke cells, `ae304f63`, `a69e9131` and `b7537006`, one each on `FreeList`, `LRU` and `Paxos`, fired first as an end-to-end smoke on the three problems that had never produced a cell and registered in advance as counting toward their conditions.

Those three cells live in a different cells root from the rest of the declared set, and the matrix root holds two landed plain-arm cells each on `FreeList`, `LRU` and `Paxos`. Three of the five problems therefore reach $n = 3$ on the plain arm only by counting one smoke cell each, and without them the registered rule takes no median on those three: the sign test becomes 2 of 2 at $p = 0.25$, which is no verdict. This is not a defect in the declaration, which names the three by identifier and says why a glob would be wrong. It is a fact about the dataset that the scoreboard is silent about, and it travels with the number. The three smoke cells also ran under an earlier export of the harness whose repairs are to the audit, carrier and canary paths and not to the task, the arm, the prompt or the caps; on that basis they contribute a price, and their containment verdicts are withdrawn rather than caveated, so no containment claim for this matrix rests on them.

Correctness: no cell of this run carries a referee verdict. Of the cells in the matrix root, 33 are priced and none carries a referee verdict on a withheld suite. The path such a verdict would be read from does not exist: five patterns swept across the whole cells root return zero files each, and the scorer reads a cost off the archive and no verdict of any kind. The only per-cell artifact that resembles a check is a manifest integrity check, which is not a correctness verdict. The withheld-suite work that does exist characterises the suites against mutants (Section 3) and is a property of the suites, not of any cell. This does not say the code was wrong. It says no instrument in this run asked, so the premium is a price for work whose correctness this run did not verify, and no sentence in this paper pairs it with quality or with working code.

Two counts in that measurement do not reconcile and are reported rather than smoothed.

Priced does not imply landed: of the 37 cells with a control directory in the matrix root, 30 landed, 3 stopped at the cost cap and are priced without having finished, and 4 failed to boot and are unpriced, which gives 33 priced. An earlier hand census in the same file reports 36 cells, 32 priced and 4 void. The file states the difference instead of adopting one number, and neither count changes the correctness answer, which is zero. Pricing a capped cell beside a landed one prices two different events under one name, and the three capped cells are named here for that reason.

The placebo arm supports nothing. An equal-cost placebo arm was run to completion on all five problems: 15 of 15 cells landed and priced, $193.42, no void cell, no cap, no drift and no failed boot. It returns UNRESOLVED on all five problems, every one inside the band $[0.4982, 2.0072]$. That outcome was the predicted one and was registered as such before any placebo cell existed, because the floor at this n is $2.0072\times$ while the treatment's own premiums on three of the five problems sit below it. The sentence this result most invites, that the placebo came in near the plain arm so the treatment's premium is method rather than form, is forbidden by name in the registration and is not written here. An arm that could have damaged the finding and did not is worth its price and is still not evidence for the finding.

A cross-stage cost claim is confounded by concurrency. The earlier two-problem stage ran mostly one cell at a time, read from its own launch receipts; this matrix ran four-wide in both its fire and its resume schedulers. The two sets of premiums therefore differ in box contention as well as in date, and nothing in the record separates those. The confound does not touch this matrix's own result, which is computed entirely within a run where both arms were measured four-wide; it bars reading the matrix's premiums as a replication of the earlier stage's magnitudes, and the placebo was fired at four-wide for the same reason. This correction was made to the campaign's own earlier claim: the two stages had been established to share a client binary, byte for byte, and were then treated as cost-comparable on that basis, which verifies one axis and generalises the verdict past it.

Every cost in this section carries an unmeasured box-load term. The concurrency confound above is a known difference between two stages. The wider one is that nothing in this campaign measures what the box was carrying while a cell was priced. The harness reaps the client and the watcher and does not reap what the subject forked: 24 processes forked by the subject of one cell in a later wave were re-parented to the init process and ran for three hours and twenty-three minutes, outliving their own cell's end by three hours and nine minutes, with 11 of the 16 priced cells of that wave metered wholly or partly inside the window. A cell's end is not the end of the cell's processes, so any cell in this campaign may have been priced on a box carrying the residue of earlier cells, and no arm looks, so no run can state its own load. The direction of that on cost is unmeasured and is not asserted here in either direction: extra processor time does not spend tokens, and a route from load to price through timeouts or extra turns was never driven. For this matrix specifically the term is bounded rather than assumed: all 37 of its cells reached their end at or before `2026-09-09T00:51:59Z` and that leak opened at `2026-09-09T03:34:51Z`, a margin of two hours, forty-two minutes and fifty-two seconds, so no cell of this matrix was metered inside it. That clears this matrix of that leak and of no other.

What this population cannot do, stated with the result. It is authored by the same people who wrote the treatment, so it carries no independent authorship and no contamination measurement of the kind a public set allows. It measures a dieted rendering of the method and not the method. It measures price and not acceptance. Three of its five magnitudes are below its own

resolvable floor and none of the five may be reported as the finding. Its gold pair is $k = 1$. And three of its five problems currently reach $n = 3$ on the plain arm by counting a cell from another root. Version 2's remedy for the last of these is under way and is described in Section 6.

5 The instrument findings, as results

Each of these was found by a gate, a probe, or a re-check in the record, and each changed the protocol.

1. **A censored cap returns the cap.** Section 4.4. Any budget derived from a capped distribution has to come from an uncensored read, or from a cap raised until the censoring fraction is near zero. Recording the state each episode had reached when the cap cut it does not rescue the estimate on its own, because under a verifier that state can carry no information: an obligation is discharged or it is not, so an incomplete proof reports zero discharged until the moment it reports all of them. A partial count is a distance and a zero count is an absent measurement, and Section 6 reports an episode that read zero at the cap and held a complete verified proof thirty-nine calls later.
2. **A statistic is a cap's price only in the cap's unit.** The quota cost per token at the higher tier was roughly double, while the token count fell; a campaign that priced the tier on tokens alone would have called it free. The metered sum and the quota unit are different quantities and a stop rule has to say which it uses.
3. **A gate can penalize the treatment for applying the treatment.** Section 2.5, three instances in one wave, the third inside the layer added to prevent the first.
4. **A screen's false positives are invisible by construction.** The campaign ran 201 scored Lean episodes and examined exactly one refusal, because exactly one existed; it was a complete proof. A defence-in-depth layer needs its own false-positive reading routinely.
5. **A blind agent that passes is a measurement of a different task.** A fence derived from the run root denied the agent its own episode directory, so the referee wrapper was unreachable; the agent wrote proofs it could never check, and two of them passed the referee at 11 to 16 times the cost of a sighted episode. The episode driver now refuses when the fence and the workspace intersect.
6. **A deny list is only as broad as the layer that enforces it.** Section 2.2. The sandbox's path denials fenced the agent's subprocesses and not the agent's own file tool, and the probe that certified the fence made its reads through the sandbox, so it could not see the layer above. A fence has to be probed in the language of every tool it is meant to bind, and the probe has to be neutral: a probe that told the agent a tool was denied and asked it to route around was answered by the agent's judgement with no tool call made, and read as blocked while measuring nothing. The audit built to find such a hole in the record had the same shape of defect: its field recorded that a call was *not blocked*, which conflates a denial, an absent file and a served read, and it reported an episode as an escape whose read had returned that the file did not exist. Only a served read is a hole being used, and a field that cannot say which of the three it saw will report the wrong one.
7. **A gate that extracts by delimiter measures the delimiter.** A statement-immutability failure was reported as the agent altering the specification; the specification was byte-identical, and the mechanism was Lean naming a cached auxiliary `match` declaration after whichever declaration elaborated it first. The audit now compares matchers by content.
8. **A guard whose predicate cannot hold looks like a guard in every review.** The recall instrument's provenance guard compared a wrapper's hash to its wrappee's and had taken the

fallback on 78 of 78 rows; the fallback read the same bytes, so no number moved. The token ceiling had been blind on every episode where it could have mattered, because the watchdog's call crashed on the first path-carrying tool call and the shell turned the crash into zero; repaired with a self-test that makes the caller's exact call.

9. **A scoreboard is silent about the structure of its own dataset.** Section 4.5. Two properties of the cost matrix are invisible in the number it produces: no cell in it carries a correctness verdict, and three of its five problems reach their registered n on the plain arm only by counting a cell fired in a different run and a different cells root. Neither is a defect in the arithmetic, and neither can be seen from the arithmetic. A result that is reported without them is a different claim from the one the data supports, so a scoreboard needs a companion statement of what its own population is made of.
10. **An instrument can be right on everything it has ever scored and wrong on the next design.** The condition key for the earlier two-arm stages is the problem and the arm. The matrix adds a third arm dimension, and a scorer keying on two fields pools the bare and statement arms of the same treatment, doubles each apparent n, and mixes the pair the design calls its gold, raising no error and producing medians that look reasonable. It was caught by building one cell and reading its control directory before building the other 56.
11. **A control arm that could only have hurt you is worth its price and is not evidence for you.** Section 4.5. The placebo arm on the systems population cost $193.42 and returned UNRESOLVED on all five problems, which was the outcome its own registration predicted from the floor at that n. It could have cleared the floor and damaged the finding; it did not. The reading that the placebo therefore supports the treatment was written down as forbidden before the arm was fired, and the question of what an arm can resolve is free before the cells and is priced at the arm's full cost afterwards.
12. **A counterfactual about someone else's instrument must be run against their code.** Section 4.3: the headline that the reference checker was sorry-inflated was refuted by its own kill-check, and what survived was a different hole.

6 The treatment question, open

Nothing in this record shows an effect of the treatment on what a referee accepts. On S2-Lean the treatment arm matched the control's set at Sonnet and was within one problem of it at Opus, with the placebo moving with the tier; the triage attributes most of the shared failures to the reference oracle. On S2-Rust and on stage C the plain arm is at or near the ceiling of every band the campaign can afford, so a comparison has no room. On S3-Systems the treatment arm cost more on all five problems under the registered sign test, and that is a difference in price carrying the four qualifiers of Section 4.5, on a run in which no cell was checked for correctness at all. A price difference is not a capability difference, and nothing in that design licenses reading one from the other. The question is open, and the next tests are:

1. **A lower tier on the hard band.** Registered as both arms at `claude-sonnet-5` on the same 13 Verus ids with the same early stop, quoted from a Sonnet probe on the band rather than from the Opus figures. The quote landed; the read was withdrawn; and the reason it was withdrawn is the result reported here. The probe paired three of the band's tasks across the two tiers, the same task and arm and instrument with only the model differing, at the same 40-call cap. Its aggregate paired token ratio was 1.55 (per-episode median 2.06), which prices the lower tier at 0.71 of the higher tier's per-episode quota and passes the affordability gate. But Sonnet passed 0 of the 3 where Opus passed 2, and all three Sonnet episodes ended at the cap, so a 13-id read

at that cap would have been in part a measurement of the cap. One further episode was run to separate the two: the same task, the same tier and the same arm, at a cap of 120 calls instead of 40.

model	cap	calls	metered tokens	wall (s)	referee at the end
`claude-opus-5`	40	23	957,722	307	9 verified, 0 errors (pass)
`claude-sonnet-5`	40	40	3,051,144	768	0 verified, 1 errors (stopped at the cap)
`claude-sonnet-5`	120	79	8,375,623	1071	10 verified, 0 errors (pass)

The 120-call episode passed cleanly: no screen violation, no helper-shape violation, the linter at return code zero, the count guard unchanged, the resource limit inside budget, the episode not void and no unblocked read of a fenced path. The cap was therefore binding at this tier on this task, and the identical Sonnet episode's report of no discharged obligations at call 40 was not a measure of how far it had to go. The planned 13-id read at 40 calls does not run, because it would return the cap. A read at 120 is a different and much larger commitment: one episode at 8.4 million tokens prices 26 episodes at roughly 218 million, and that figure is a lower bound, since the episode it is priced from passed at 79 of its 120 calls on the task the higher tier found easiest of the three. The two-tier comparison is therefore open and belongs to version 2. What the pair does establish is narrower, and it is one episode against one: on this task the lower tier reaches the same referee verdict as the higher one, and pays 8.7 times the tokens and 3.4 times the calls to get there.

2. **A CLEVER variant with the oracle repaired.** Replace the isomorphism obligation with an implication-with-witness (the agent's specification implies the human one, and the reference implementation satisfies the agent's), exclude problem 18, and rerun the seven problems both arms failed. This changes what the benchmark measures and is registered as a variant, not as a correction to the published Task 1 numbers.
3. **The systems population, with the four things it is missing.** The custom population of Section 3 exists, is frozen, and has produced the cost reading of Section 4.5. Four registered gaps stand between that reading and a capability comparison on the same population, and each is a separate piece of work. First, a correctness instrument: a referee verdict per cell on the withheld suite, written to a path the scorer reads, so that a premium can be paired with an outcome rather than with a price alone. Second, three plain-arm cells inside the matrix's own cells root on `FreeList`, `LRU` and `Paxos`, so that all five problems reach $n = 3$ within one run and the sign test can be reported both with and without the smoke cells; if the two readings agree the dependency is discharged, and if they diverge the divergence is the result and is reported ahead of the headline. Third, a `## Statement` section authored for the four cards that lack one, which is the only thing standing between the gold pair and $k = 5$; it is an author's hour per card and it decides what the treatment arm measures, so it is not the harness's to invent. Fourth, an arm carrying the method as written rather than the dieted rendering, which the current cost cap refused once already. A scout for a second population over a contamination-free Rust software-engineering set returned no: of its 113 hand-authored tasks 5 are Rust, below the registered threshold of 8 before any screening, and none of the 5 survived the screen for a specification layer. Version 2's populations ship in the Harbor task format [6], so that the referee, the fence and the task remain one object.

The failure surface on S2-Lean is also a finding about the benchmark: four of the twelve remaining stage-C oracles are permissive on regions their tests never visit, and the four-label triage taxonomy has no cell for a reference specification that contradicts its own tests. The reproduction scripts for both are in the repository, for the benchmark's authors; the disclosure itself is a separate act and is not claimed here.

7 Reproducibility

Every pin is a line in `harness/HASHES.txt`: the CLEVER commit, the Lean toolchain and mathlib revision, the project manifest and lakefile hashes, the Verus release and its rust channel, the z3 [4], vstd and lynette hashes, the resource limit and seed, every prompt, arm file, checker and driver, and the set-hash of the 207 rebuilt Verus views. The evaluation images for the SWE-bench substrate are pinned by digest in `IMAGE-DIGESTS.json`. The episode script refuses to run a stage whose view or checker hash is not the pinned one, and each manifest records the freeze commit, the hashes it asserted, the model id read from every response, and the transcript's hash. The morning-line instruments that produced every rate in this paper are in the repository with their self-tests, and the evidence directories carry their outputs byte for byte.

The systems population is pinned differently, because it is authored here rather than derived from a public source. Its five problems are frozen in a dated export, the scored set is declared by cell identifier rather than by a pattern over the archive, each cell's control directory records its task, arm and card extras, and each cell's price is read from its own harvested meter file. The client is pinned by absolute versioned path in the registration, not resolved from the shell path. One gap in that chain is reported rather than closed: the matrix's cells carry no build-provenance record of their own, so the mapping from cell to instrument set is a reconstruction after the fact and not a receipt, and the claim that this matrix and the earlier stage ran the same client rests on a byte-size comparison against the earlier stage's own record.

The task populations are re-derived from the pinned sources by the harness's view builders; the repository redistributes only what `PROVENANCE.md` lists. The run records and the S2 episode archives are a separate data asset; its DOI is assigned at release (Zenodo) and recorded in the repository on the flip day. Version 2's populations ship as Harbor tasks so that the referee, the fence and the task are one object.

The code is released under Apache-2.0 and the data and documents under CC BY 4.0. The agent was Claude Code [1] on a consumer subscription, and the transcripts are published as this campaign's own measurements: the provider's Consumer Terms effective 2025-10-08 [2] assign outputs to the user and do not restrict their publication, and the training restriction in the Usage Policy dated 2025-09-15 [3] binds the subscriber rather than a recipient of these files. Both documents are cited at the version read, because both are revised in place.

Acknowledgements

CLEVER is by the Trishul group at UT Austin (MIT licence); VeruSAGE-Bench and lynette are by Microsoft (MIT licence); SWE-bench Verified is by the SWE-bench authors with OpenAI's verification pass; the source repositories of the drawn issues carry their own licences, listed in `PROVENANCE.md`. The five systems components of S3, their withheld suites and their mutant sets were authored for this benchmark by the author's assistant heads under the author's direction, which is the independence limitation stated in Section 4.5. The runs were executed by Claude Code with Anthropic's Sonnet 5 and Opus 5 models. The protocol documents, amendments and result files in the repository were written by the author's assistant heads under the author's direction and carry the author's responsibility.

References

[1] Anthropic. *Claude Code*, version 2.1.251. `https://code.claude.com/docs/en/overview`.

[2] Anthropic. *Consumer Terms of Service*, effective October 8, 2025. https://www.anthropic.com/legal/consumer-terms.

[3] Anthropic. *Usage Policy*, effective September 15, 2025. https://www.anthropic.com/legal/aup.

[4] Leonardo de Moura and Nikolaj Bjørner. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008)*, pages 337–340, 2008. https://doi.org/10.1007/978-3-540-78800-3_24.

[5] Leonardo de Moura and Sebastian Ullrich. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction — CADE 28*, pages 625–635, 2021. https://doi.org/10.1007/978-3-030-79876-5_37.

[6] Harbor Framework Team. *Harbor: A framework for evaluating and optimizing agents and models in container environments*. Zenodo, 2026. https://doi.org/10.5281/zenodo.20953922.

[7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770. https://arxiv.org/abs/2310.06770.

[8] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. Verus: Verifying Rust Programs using Linear Ghost Types. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1):286–315, 2023. https://doi.org/10.1145/3586037.

[9] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381, 2020. https://doi.org/10.1145/3372885.3373824.

[10] Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018. https://doi.org/10.1073/pnas.1708274114.

[11] Princeton NLP. *SWE-bench Verified*. Hugging Face dataset, split `test`, 500 rows; revision `c104f840` as pinned here. The dataset card carries no licence field. https://huggingface.co/datasets/princeton-nlp/SWE-bench_Verified.

[12] Alessandro Sosso, Akhil Arora, and Bas Spitters. Agentic Proving for Program Verification. arXiv:2605.23772, 2026. https://arxiv.org/abs/2605.23772.

[13] Xudong Sun, Wenjie Ma, Jiawei Tyler Gu, Zicheng Ma, Tej Chajed, Jon Howell, Andrea Lattuada, Oded Padon, Lalith Suresh, Adriana Szekeres, and Tianyin Xu. Anvil: Verifying Liveness of Cluster Management Controllers. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2024)*, pages 649–666. USENIX Association, 2024. https://www.usenix.org/conference/osdi24/presentation/sun-xudong.

[14] Amitayush Thakur, Jasper Lee, George Tsoukalas, Meghana Sistla, Matthew Zhao, Stefan Zetzsche, Greg Durrett, Yisong Yue, and Swarat Chaudhuri. CLEVER: A Curated Benchmark for Formally Verified Code Generation. arXiv:2505.13938, 2025. https://arxiv.org/abs/2505.13938.

[15] Chenyuan Yang, Natalie Neamtu, Chris Hawblitzel, Jacob R. Lorch, and Shan Lu. VeruSAGE: A Study of Agent-Based Verification for Rust Systems. arXiv:2512.18436, 2025. `https://arxiv.org/abs/2512.18436`.

A S2-Lean U15, per problem

problem	a0 S	a1 S	a2 S	a0 O	a2 O	a1 O
73	F	P	F	P	P	P
0	F	F	F	F	F	F
146	P	P	P	P	P	P
16	P	P	P	P	P	P
4	P	F	P	F	P	F
38	P	P	P	P	P	P
142	P	P	P	P	P	P
96	F	F	F	F	F	F
112	F	F	F	P	F†	F
141	F	F	F	P	P	P
31	P	P	P	P	P	P
54	P	P	P	P	P	P
127	F	F	F	F	F	F
18	F	F	F	F	F	F
74	P	P	P	P	P	P
proven	8	8	8	10	10	9

S = `claude-sonnet-5`, O = `claude-opus-5`, stage B, 100-call cap. † refused by the pragma screen; a registered re-check over the frozen artifact found a complete proof (diagnostic 11 of 15).

B The stage-B failure triage

problem	label	ground, in one line
0	HUMAN-LOOSE	conclusion guarded by `numbers.length > 1`; vacuous on short lists; witness recorded
4	HUMAN-LOOSE	guarded by `0 < numbers.length`; vacuous on `[]`
18	AGENT-WRONG	agents scan `List.range (|s| - |t| + 1)`, contradicting a `#test`; the human spec is also unsatisfiable
73	PROOF-HARD	equivalent specs; the equivalence needs an optimality argument
96	HUMAN-LOOSE	spec fixes membership only, never order or multiplicity; witness `primes ++ primes`
112	PROOF-HARD	true isomorphism; one sibling cell proved it completely
127	HUMAN-LOOSE	guarded by interval well-formedness; vacuous on ill-formed intervals
141	PROOF-HARD	reduces to a `String.splitOn` equivalence; 7 to 8 kB of lemmas written and still out of rounds

Totals over 30 triageable cells: HUMAN-LOOSE 17, AGENT-WRONG 5, PROOF-HARD 8; one screen-refused cell excluded.

C The S2-Rust hard band

Upper tercile of the 180 live tasks by reference-proof wall, rank ≥ 120; band $n = 60$, Anvil-Advanced 42 and NRKernel 18; band wall median 4.55 s, max 358.60 s; band bytes median 165,568. Draw of 13 at seed 20260902: 11 Anvil-Advanced, 2 NRKernel. Ten run under the sequential stop: 9 `PASS`, 1 `VERIFY_FAIL` at the cap; sighted episodes reach the first clean referee run at a median of 2 referee calls (p90 9). The 9 is a lower bound on the band's pass count at this cap: the failed episode ended at 40 of 40 calls one obligation short (8 verified, 1 error), the uncensored passing call counts run 23 to 39 against the cap of 40 with p90 = 39, and one pass landed at exactly 40 of 40.